

**AICTE - EDUSKILLS VIRTUAL INTERNSHIP
PROJECT REPORT**

"Text Search Indexer"

SUBMITTED BY:

Student Name: ALLADI SUSMITHA

Roll Number: 323103282002

Program: B.Tech

Branch: AI-ML

Semester: Unknown

Academic Year: None-2027

SUBMITTED TO:

Gayatri Vidya Parishad College of Engineering for Women

Internship Details:

AICTE Student ID: STU678ccb4bc76ee1737280331

Internship Duration: 10 Weeks (Jan 2026 - Mar 2026)

Supported by: Eduskills Academy

Under the Guidance of

Project Manager

Eduskills

TO WHOMSOEVER IT MAY CONCERN

This is to certify that **Mr./Ms. ALLADI SUSMITHA**, Roll No. **323103282002**, a student of **Gayatri Vidya Parishad College of Engineering for Women**, has successfully completed **10-Week (Jan 2026 - Mar 2026)** AICTE – EduSkills Virtual Internship Program.

During the internship, he/she worked on the project titled **"Text Search Indexer"**. His/her performance during the internship was found to be satisfactory, and he/she demonstrated good technical and professional skills.

We wish him/her success in future endeavors.

Sagar Bala Behera

Mentor, EduSkills Academy

Date: 04-03-2026

DECLARATION

I hereby declare that the work presented in this Online Virtual Internship Project Report is my original work carried out under the guidance of my industry mentor and faculty guide. This work has not been submitted earlier for the award of any degree or diploma.

ALLADI SUSMITHA

(Signature)

Date: 04-03-2026

ACKNOWLEDGEMENT

I express my sincere gratitude to **AICTE & EduSkills** for providing me with the opportunity to undertake this online virtual internship. I would like to thank my industry mentor **Mr./Ms. Sagar Bala Behera** for his/her valuable guidance, continuous support, and technical insights throughout the internship period.

I am also thankful to the Department of **AI-ML** for their encouragement and support.

ABSTRACT

This project centers on the development of a comprehensive Text Search Indexer in Java. The system is designed to process textual data, including normalization of tokens, building word frequency maps, and implementing fundamental search functionalities. The objective is to simulate core components of search engine technology, from data preprocessing to information retrieval, providing a foundational understanding of indexing and searching principles.

Table of Contents

1. Introduction	2
2. Organization Profile	3
3. Internship Overview	4
4. Problem Statement	5
5. Methodology	6
6. Tools & Technologies	7
7. Implementation	8
8. Learning Outcomes	11
9. Challenges & Solutions	12
10. Conclusion	13
11. Future Scope	14
12. References	15
13. Annexure	16

1. Introduction

Industry-oriented learning through internships plays a crucial role in bridging the gap between academic knowledge and real-world applications. This online virtual internship provided hands-on exposure to **Java Full Stack With Project** and its application in modern software solutions.

2. Organization Profile

Organization Name: EduSkills Academy

Domain: EdTech & Skill Development

Services: Virtual Internships

Vision: To empower students with industry-relevant skills.

3. Internship Overview

- Internship Mode: **Online / Virtual**
- Duration: **10 Weeks**
- Platform Used: **EduSkills LMS**
- Role: **Intern**

4. Problem Statement & Objectives

Problem Statement:

The Text Search Indexer is a Java-based application designed to create an index for text documents, enabling efficient searching. This project encompasses several key stages of search engine development, starting with the preprocessing of textual data. It includes robust text normalization to standardize input, followed by tokenization to break down text into manageable units. The system further identifies unique terms and calculates term frequencies to build a foundational index. The core functionality is augmented by a keyword matching component, allowing for basic search queries. This project aims to provide a practical insight into the internal workings of text search systems, from data ingestion to retrieval.

Objectives:

- To design and implement a solution for the given problem statement.
- To utilize modern tools and algorithms.
- To optimize performance and code quality.

5. Project Description & Methodology

The development of the Text Search Indexer was approached in a step-by-step, modular fashion, addressing each component of the search pipeline sequentially. First, the Text Normalization module was implemented to convert all input text to lowercase and remove punctuation, ensuring consistency. Following this, the Basic Tokenizer was developed to split normalized text into individual words. The Unique Word Indexer then processed these tokens to identify and store distinct words. Subsequently, the Keyword Search Matcher was integrated to verify the presence of specific keywords within lines. Finally, the Term Frequency Counter was built to calculate the occurrence of each word in a given text, contributing to the overall indexing and search capabilities.

6. Tools & Technologies Used

Programming Language: Java

Core Java Libraries: Standard Java I/O, Collections Framework (e.g., HashMap, ArrayList), String manipulation methods.

Concepts: Text Processing, Tokenization, Normalization, Data Structures (Maps, Sets), Algorithms (Frequency Counting), Search Indexing Fundamentals.

7. Implementation & Results

Problem 1: Text Normalization

Description: The objective of Text Normalization is to preprocess raw text data by converting all characters to lowercase and removing non-alphanumeric characters, thereby ensuring that variations in casing and punctuation do not affect search results or data processing. This step is crucial for creating a consistent and unified representation of the text.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();

            if (line.trim().isEmpty()) {
                results.add("{\"normalized\":[ ]}");
            } else {
                String normalized = line
                    .toLowerCase()
                    .replaceAll("[^a-z0-9 ]", "");

                String escaped = normalized
                    .replace("\\", "\\")
                    .replace("\"", "\\");

                results.add("{\"normalized\":[\"" + escaped + "\"]}");
            }
        }

        scanner.close();
        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 2: Basic Tokenizer

Description: The Basic Tokenizer module is responsible for segmenting normalized text into individual words or tokens. This process involves splitting the input string based on whitespace delimiters, resulting in a structured list of tokens that can be further processed for indexing and analysis.

```

import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();

            if (line.trim().isEmpty()) {
                results.add("{\"tokens\":[]}");
            } else {
                String[] tokens = line.trim().split("\\s+");

                StringBuilder sb = new StringBuilder();
                sb.append("{\"tokens\":[");

                for (int i = 0; i < tokens.length; i++) {
                    String escaped = tokens[i]
                        .replace("\\", "\\")
                        .replace("\"", "\\");

                    sb.append("\"").append(escaped).append("\"");
                    if (i < tokens.length - 1) {
                        sb.append(",");
                    }
                }

                sb.append("]");
                results.add(sb.toString());
            }
        }

        scanner.close();
        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}

```

Problem 3: Unique Word Indexer

Description: The Unique Word Indexer's task is to identify and extract all distinct words from a given set of tokens. This involves leveraging data structures that automatically handle uniqueness, such as sets, to efficiently collect all unique terms present in the text, forming a vocabulary for the index.

```

import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();

            if (line.trim().isEmpty()) {
                results.add("{\"unique\":[]}");
            } else {
                String[] words = line.trim().split("\\s+");
                TreeSet<String> uniqueWords = new TreeSet<>();

                for (String word : words) {
                    uniqueWords.add(word);
                }

                StringBuilder sb = new StringBuilder();
                sb.append("{\"unique\":[");

                int count = 0;
                for (String word : uniqueWords) {
                    String escaped = word
                        .replace("\\", "\\")
                        .replace("\"", "\\");

                    sb.append("\"").append(escaped).append("\"");
                    if (count < uniqueWords.size() - 1) {
                        sb.append(",");
                    }
                    count++;
                }

                sb.append("]");
                results.add(sb.toString());
            }
        }

        scanner.close();
        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}

```

Problem 4: Keyword Search Matcher

Description: The Keyword Search Matcher functionality is designed to perform a simple check for the presence of a specified keyword within a given line of text. It returns a boolean indicator (represented as a string 'true' or 'false') to signify whether the keyword exists in the processed text.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();

            if (line.toLowerCase().contains("index")) {
                results.add("{\"match\": \"true\"}");
            } else {
                results.add("{\"match\": \"false\"}");
            }
        }

        scanner.close();
        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

Problem 5: Term Frequency Counter

Description: The Term Frequency Counter aims to quantify the occurrence of each word within a line of text. It processes the tokens generated from a line and counts the frequency of each unique term, providing a summary output that details word counts, which is fundamental for ranking search results.

```
import java.util.*;

public class Solution {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        List<String> results = new ArrayList<>();

        while (scanner.hasNextLine()) {
            String line = scanner.nextLine();

            String[] words = line.toLowerCase().split("\\s+");
            int count = 0;

            for (String word : words) {
                if (word.equals("the")) {
                    count++;
                }
            }

            results.add("{\\\"summary\\\": [\\\"the:\" + count + \"\\\"]}");
        }

        scanner.close();
        System.out.print("[");
        for (int i = 0; i < results.size(); i++) {
            System.out.print(results.get(i));
            if (i < results.size() - 1) System.out.print(",");
        }
        System.out.println("]");
    }
}
```

8. Learning Outcomes

- Gained practical experience in implementing core text processing techniques in Java, including normalization and tokenization.
- Developed a foundational understanding of data structures like HashMaps for building frequency counts and managing indexes.
- Learned to systematically approach complex problems by breaking them down into smaller, manageable modules.
- Acquired skills in designing and implementing algorithms for text analysis and search.

9. Challenges Faced & Solutions

Challenge: Handling diverse punctuation and special characters during text normalization. Solution: Implemented regular expressions to systematically identify and remove a wide range of non-alphanumeric characters while preserving word integrity.

Challenge: Efficiently counting term frequencies for large volumes of text. Solution: Utilized Java's HashMap data structure to store word counts, allowing for $O(1)$ average time complexity for updates and lookups, thereby optimizing the counting process.

10. Conclusion

The Text Search Indexer project successfully demonstrates the fundamental principles of building a text processing and search system. By implementing modules for normalization, tokenization, indexing, and basic search, the project provides a solid foundation for understanding how search engines process and retrieve information. The developed system offers a practical example of applying Java programming to solve real-world data analysis challenges.

11. Future Scope

- Enhance the search functionality to support phrase searching and boolean operators (AND, OR, NOT).
- Implement more sophisticated ranking algorithms, such as TF-IDF, to improve search result relevance.
- Extend the indexer to support searching across multiple documents or files concurrently.

12. References

Java Documentation (Oracle): <https://docs.oracle.com/javase/8/docs/>

Java

Collections

Framework

Overview:

<https://docs.oracle.com/javase/8/docs/technotes/guides/collections/>

Regular Expressions in Java: <https://docs.oracle.com/javase/tutorial/essential/regex/>

13. Annexure

- Internship Completion Certificate
- Offer Letter